TP 7
Les objets en JavaScript : Math, String, Date, tableaux et objets utilisateurs

Filière / Niveau	Licence Informatique — 1ère année
Module	Développement Web
Thème 2	JavaScript et Bootstrap
Durée	1 séance (2 h 30) + 1 exercice à terminer à la maison (45 min)
Prérequis	TP6 (console, variables, conversions, fonctions, fonctions fléchées), cycle Git add → commit → push

Objectifs du TP

À l'issue de ce TP, l'étudiant sera capable de :

- Utiliser les méthodes les plus utiles des objets **Math**, **String** et **Date** à partir de l'aide-mémoire (partie I).
- Créer et manipuler un **tableau** : indices, length, push, pop, includes, sort, join, et le parcourir avec for...of.
- Transformer et résumer un tableau avec map, filter et reduce (puis find, some, every à la maison).
- Définir un **objet utilisateur** (propriétés, une méthode avec this), le parcourir avec for...in, et travailler avec un **tableau d'objets**.

I. Aperçu théorique

1.1 La notion d'objet

En C, une struct regroupe des données ; en JavaScript, un **objet** regroupe des données (**propriétés**) et des fonctions qui agissent sur elles (**méthodes**). On accède aux deux avec le **point** : objet.propriete, objet.methode(). JavaScript fournit des objets prêts à l'emploi (Math, String, Date, Array, et console déjà rencontré au TP6) ; on peut aussi créer ses propres objets, dits objets utilisateurs (voir 1.6).

Règle : une propriété se lit sans parenthèses, une méthode s'appelle avec. Math.round sans parenthèses ne calcule rien : il désigne la fonction elle-même.

1.2 L'objet Math

Math est un objet **statique** : ses propriétés et ses méthodes s'appellent directement à partir de Math (Math.PI, Math.round(x)), sans créer d'objet avec new. Il remplace math.h du C.

Méthode	Rôle	Exemple
Math.round(x)	arrondi à l'entier le plus proche	console.log(Math.round(2.5)) → 3

Méthode	Rôle	Exemple
		console.log(Math.round(2.4)) → 2
Math.floor(x) Math.ceil(x)	arrondi vers le bas / vers le haut	console.log(Math.floor(7.8)) → 7 console.log(Math.ceil(7.2)) → 8
Math.trunc(x)	supprime la partie décimale	console.log(Math.trunc(-7.8)) → -7 (floor donnerait -8)
Math.abs(x)	valeur absolue	console.log(Math.abs(-3)) → 3
Math.max(a, b, ...) Math.min(a, b, ...)	le plus grand / le plus petit	console.log(Math.max(3, 9, 4)) → 9
Math.pow(x, y) Math.sqrt(x)	puissance (comme **) / racine carrée	console.log(Math.sqrt(16)) → 4 console.log(Math.sqrt(-1)) → NaN
Math.random()	nombre décimal aléatoire dans [0 ; 1[	console.log(Math.random()) → 0.7312... (change à chaque appel)
Math.PI	propriété : la constante π	console.log(Math.PI) → 3.141592653589793

1.3 Les chaînes de caractères (String)

Une chaîne **ne peut pas être modifiée sur place** : aucune méthode ne la change, chacune **renvoie une nouvelle chaîne** qu'il faut ranger dans une variable. Le premier caractère est à l'indice 0, comme en C. Exemples avec const s = "Club Informatique" :

Propriété / méthode	Rôle	Exemple
chaine.length	nombre de caractères (propriété)	console.log(s.length) → 17
chaine[i] chaine.charAt(i)	caractère à l'indice i	console.log(s[0]) → C console.log(s.charAt(5)) → I
chaine.toUpperCase() chaine.toLowerCase()	en majuscules / en minuscules	console.log(s.toUpperCase()) → CLUB INFORMATIQUE
chaine.trim()	supprime les espaces au début et à la fin	console.log(" Ali ".trim()) → Ali
chaine.indexOf(ch)	indice de la première occurrence, -1 si absente	console.log(s.indexOf("Info")) → 5 console.log(s.indexOf("PHP")) → -1
chaine.includes(ch)	true si la chaîne contient ch	console.log(s.includes("Club")) → true

Propriété / méthode	Rôle	Exemple
chaine.startsWith(ch) chaine.endsWith(ch)	commence / se termine par ch	console.log(s.startsWith("Club")) → true
chaine.slice(debut, fin)	sous-chaîne de debut (inclus) à fin (exclu) ; fin facultatif	console.log(s.slice(0, 4)) → Club console.log(s.slice(5)) → Informatique
chaine.substring(debut, fin)	comme slice (forme utilisée dans le cours), mais sans indices négatifs	console.log(s.substring(0, 4)) → Club console.log(s.slice(-4)) → ique
chaine.replace(a, b)	remplace la première occurrence de a par b	console.log(s.replace(" ", "-")) → Club-Informatique
chaine.split(sep)	découpe en tableau selon le séparateur	console.log(s.split(" ")) → ["Club", "Informatique"]
chaine.repeat(n)	répète la chaîne n fois	console.log("-".repeat(5)) → -----

1.4 L'objet Date

Contrairement à Math, un objet Date se **crée** avec new. Sans argument, il contient la date et l'heure de l'instant. **Attention** : les mois sont numérotés de **0** (janvier) à **11** (décembre), et getDay() renvoie le jour de la semaine de 0 (dimanche) à 6 (samedi). Exemples avec const reunion = new Date(2026, 11, 15, 14, 30) :

Méthode	Rôle	Exemple
new Date()	date et heure de l'instant	const maintenant = new Date();
new Date(annee, mois, jour, h, min)	une date précise (**mois de 0 à 11**)	const reunion = new Date(2026, 11, 15, 14, 30); → 15 décembre 2026 à 14 h 30
date.getFullYear()	année sur 4 chiffres (getYear() est obsolète)	console.log(reunion.getFullYear()) → 2026
date.getMonth()	mois, de 0 (janvier) à 11 (décembre)	console.log(reunion.getMonth()) → 11
date.getDate()	jour du mois, de 1 à 31	console.log(reunion.getDate()) → 15
date.getDay()	jour de la semaine, de 0 (dimanche) à 6 (samedi)	console.log(reunion.getDay()) → 2 (mardi)
date.getHours() date.getMinutes() date.getSeconds()	heure, minutes, secondes	console.log(reunion.getHours()) → 14 console.log(reunion.getMinutes())

Méthode	Rôle	Exemple
		→ 30
date.getTime()	millisecondes écoulées depuis le 1er janvier 1970 (sert à calculer une durée entre deux dates)	console.log(reunion.getTime()) → 1797341400000
date.toLocaleDateString("fr-FR")	la date au format français	console.log(reunion.toLocaleDateString("fr-FR")) → 15/12/2026
date.toLocaleTimeString("fr-FR")	l'heure au format français	console.log(reunion.toLocaleTimeString("fr-FR")) → 14:30:00

```
const maintenant = new Date();                // date et heure actuelles
const reunion = new Date(2026, 11, 15, 14, 30);  // 15 décembre 2026 à 14 h 30 (11 = décembre)
console.log(maintenant.getFullYear());        // 2026
console.log(reunion.getMonth());              // 11
console.log(reunion.getDate());               // 15 (jour du mois)
console.log(reunion.getDay());                // 2 (mardi : 0 = dimanche)
console.log(reunion.getHours(), reunion.getMinutes()); // 14 30
console.log(reunion.toLocaleDateString("fr-FR")); // "15/12/2026"
```

1.5 Les tableaux (Array)

a) Création, indices, length et méthodes principales

Un tableau se crée entre **crochets**. Contrairement au C, sa taille n'est pas fixée à l'avance : il peut grandir ou rétrécir. Le premier élément est à l'indice **0**.

Exemples avec const ateliers = ["HTML", "CSS", "JavaScript"] :

Écriture / méthode	Rôle	Exemple
[v1, v2, ...]	création	const ateliers = ["HTML", "CSS", "JavaScript"];
tableau[i]	élément d'indice i (à partir de 0)	console.log(ateliers[0]) → HTML
tableau.length	nombre d'éléments (propriété, sans parenthèses)	console.log(ateliers.length) → 3
tableau[tableau.length - 1]	le dernier élément	console.log(ateliers[ateliers.length - 1]) → JavaScript

Écriture / méthode	Rôle	Exemple
tableau[i] = v	remplace l'élément d'indice i	ateliers[1] = "CSS3"; → le tableau devient ["HTML", "CSS3", "JavaScript"]
tableau.push(v) tableau.pop()	ajoute v à la fin / retire et renvoie le dernier	ateliers.push("Git"); → ajoute "Git" à la fin console.log(ateliers.pop()) → Git (retiré du tableau)
tableau.unshift(v) tableau.shift()	même chose au début du tableau	ateliers.unshift("Web"); → ajoute "Web" au début console.log(ateliers.shift()) → Web (retiré du tableau)
tableau.indexOf(v) tableau.includes(v)	indice de v (-1 si absent) / true si v est présent	console.log(ateliers.indexOf("CSS")) → 1 console.log(ateliers.includes("PHP")) → false
tableau.join(sep)	renvoie une chaîne des éléments séparés par sep (inverse de split)	console.log(ateliers.join(" - ")) → HTML - CSS - JavaScript
tableau.concat(tableau2)	renvoie un nouveau tableau formé des deux	console.log(ateliers.concat(["Git"])) → ["HTML", "CSS", "JavaScript", "Git"]
tableau.slice(debut, fin)	renvoie une copie partielle (fin exclu)	console.log(ateliers.slice(0, 2)) → ["HTML", "CSS"]
tableau.splice(i, n, ...v)	à partir de l'indice i, supprime n éléments et insère les valeurs v (facultatives)	ateliers.splice(1, 1); → retire "CSS" : ["HTML", "JavaScript"]
tableau.sort()	trie **comme des chaînes** : [10, 9, 2] devient [10, 2, 9] ! Pour des nombres : tableau.sort((a, b) => a - b)	ateliers.sort(); → le tableau devient ["CSS", "HTML", "JavaScript"]
tableau.reverse()	inverse l'ordre	ateliers.reverse(); → le tableau devient ["JavaScript", "CSS", "HTML"]
console.log(tableau)	affiche le tableau sur une ligne (fonction de la **console**, pas méthode du tableau)	console.log(ateliers) → ["HTML", "CSS", "JavaScript"]
console.table(tableau)	affiche le tableau en **grille** indice / valeur (une colonne par propriété pour un tableau d'objets)	console.table(ateliers) → grille : 0 HTML / 1 CSS / 2 JavaScript

Remarque : lire un indice inexistant ne provoque pas d'erreur, ateliers[10] → undefined. Avec const, on ne peut pas réaffecter la variable (ateliers = [...] est une erreur), mais on peut modifier le **contenu** du tableau (push, pop, t[i] = v...).

b) Parcourir un tableau

Trois écritures, du C vers le JavaScript moderne.

for...of donne directement **chaque valeur** ;

forEach reçoit une **fonction** (souvent fléchée) appelée pour chaque élément, avec l'élément puis, si on le demande, son indice.

```
for (let i = 0; i < ateliers.length; i++) {   // comme en C : utile quand on a besoin de i
  console.log(i, ateliers[i]);
}
for (const atelier of ateliers) {            // chaque valeur, dans l'ordre
  console.log(atelier);
}
ateliers.forEach((atelier, i) => {           // fonction appelée pour chaque élément
  console.log(i + 1 + ". " + atelier);
});
```

Remarque (convention) : un tableau porte un nom au **pluriel** (ateliers) et un de ses éléments le même nom au **singulier** (atelier). Attention au « s » !

c) Transformer un tableau : map, filter, reduce, find, some, every

Pour calculer quelque chose à partir d'un tableau (une somme, une sélection, une transformation), on écrit d'habitude une boucle for qui remplit une variable petit à petit.

Les méthodes ci-dessous font cette boucle à notre place : on leur donne une **fonction fléchée** qui dit quoi faire de chaque élément, et elles renvoient le résultat dans une **nouvelle** variable, sans toucher au tableau d'origine.

Exemples avec const notes = [12, 8, 15, 9.5, 17] :

fn est la fonction fléchée ; n est l'élément en cours (ici une note).

Méthode	Rôle	Exemple
map(fn)	transforme chaque élément et renvoie le nouveau tableau (même taille)	console.log(notes.map((n) => n + 1)) → [13, 9, 16, 10.5, 18]
filter(fn)	garde les éléments qui vérifient la condition, dans un nouveau tableau	console.log(notes.filter((n) => n >= 10)) → [12, 15, 17]
reduce(fn, init)	**réduit** le tableau à une seule valeur, en combinant les éléments un à un (ici une somme : acc = total en cours, qui part de init = 0)	console.log(notes.reduce((acc, n) => acc + n, 0)) → 0 + 12 + 8 + 15 + 9.5 + 17 = 61.5

Méthode	Rôle	Exemple
find(fn)	renvoie le premier élément qui vérifie la condition, sinon undefined	console.log(notes.find((n) => n < 10)) → 8
some(fn)	true si au moins un élément vérifie la condition	console.log(notes.some((n) => n === 20)) → false
every(fn)	true si tous les éléments vérifient la condition	console.log(notes.every((n) => n >= 5)) → true

1.6 Les objets utilisateurs

a) Propriétés

Un objet se crée entre **accolades**, sous forme de paires propriete: valeur séparées par des virgules. Exemples avec const membre = { prenom: "Sarra", age: 19 } :

Écriture	Rôle	Exemple
{ propriete1: valeur1, propriete2: valeur2, ... }	création : chaque propriété s'écrit propriete: valeur ; les propriétés sont séparées par des virgules. (La documentation officielle parle de « clé : valeur » : une clé n'est ici que le **nom** d'une propriété.)	const membre = { prenom: "Sarra", age: 19 };
propriete: [...]	une propriété peut contenir un tableau (ou un objet)	const membre = { prenom: "Sarra", ateliers: ["HTML", "CSS"] }; console.log(membre.ateliers[0]) → HTML
objet.propriete	lire une propriété (point)	console.log(membre.prenom) → Sarra
objet.propriete = valeur	modifier une propriété	membre.age = 20; → la propriété age vaut maintenant 20
objet["propriete"]	même chose avec des crochets et le nom entre guillemets	console.log(membre["age"]) → 19
objet.nouvellePropriete = valeur	ajoute une propriété avec sa valeur (il suffit de l'affecter)	membre.email = "sarra@club.tn"; → membre a maintenant trois propriétés : prenom, age, email
delete objet.propriete	supprime une propriété	delete membre.email; → membre revient à deux propriétés : prenom, age
objet.inexistante	propriété absente : pas d'erreur	console.log(membre.telephone) → undefined

Écriture	Rôle	Exemple
for (const propriete in objet)	parcourt les **noms** des propriétés (accès par crochets)	for (const p in membre) console.log(p, membre[p]); → prenom Sarra / age 19
Object.keys(objet)	tableau des noms des propriétés	console.log(Object.keys(membre)) → ["prenom", "age"]
console.log(objet)	affiche l'objet (à déplier dans la console)	console.log(membre) → {prenom: "Sarra", age: 19}

b) Méthodes et this

Une **méthode** est une fonction définie à l'intérieur d'un objet : une action que l'objet sait faire.

this désigne **l'objet lui-même** : c'est ainsi que la méthode accède aux propriétés de l'objet.

Dans ce TP, une méthode s'écrit avec la forme courte nom() { ... } (la forme nom: function () { ... } du cours est équivalente).

Remarque — quand utiliser une fonction fléchée ? Pour les fonctions de rappel de map, filter, forEach... Jamais pour une méthode : une fonction fléchée n'a pas son propre this, donc this.prenom ne désignerait pas l'objet.

```
const membre = {
  prenom: "Sarra",
  ateliers: ["HTML", "CSS"],
  sePresenter() {                    // méthode (forme courte)
    return "Je suis " + this.prenom + ", " + this.ateliers.length + " ateliers.";
  },
  inscrire(atelier) {
    this.ateliers.push(atelier);
  }
};
membre.inscrire("JavaScript");
console.log(membre.sePresenter());  // "Je suis Sarra, 3 ateliers."
```

c) Parcourir un objet : for...in

for...in parcourt les **clés** d'un objet (voir tableau ci-dessus) ; for...of parcourt les **valeurs** d'un tableau. Ne pas les confondre : sur un tableau, for...in donnerait les indices.

II. Application pratique

Dans ce TP, le dossier et le dépôt s'appellent TP7. Le TP comprend **un exercice en classe**, « Le club informatique », en quatre parties courtes qui suivent l'ordre de la partie I (Math et

String, Date, tableaux, objets), et **un exercice de synthèse à terminer à la maison**. Comme au TP6, la page ne contient qu'un titre : **tout se passe dans la console**.

Déroulement de la séance : correction du TP6 (20 min), présentation de la partie I (40 min), préparation Git (5 min), exercice 1 (65 min).

Consignes

a. Le code JavaScript s'écrit dans le fichier du dossier js ; la page relie son script dans le <head>, avec defer.

b. La console reste ouverte pendant tout le TP : après chaque modification, enregistrer (Ctrl + S), actualiser (F5) et vérifier qu'aucune erreur rouge n'apparaît. L'affichage se fait avec console.log (pas de document.write, ni d'alert pour afficher un résultat).

c. Les variables se déclarent avec const (ou let si la valeur change) ; jamais var, jamais sans mot-clé. Les tableaux et les objets se déclarent avec const : on modifie leur contenu, jamais la variable.

d. Les tableaux s'écrivent entre crochets [ ] et les objets entre accolades { } (pas de new Array(), new Object()). Les fonctions de rappel de map, filter, reduce s'écrivent en **fonctions fléchées** ; les méthodes des objets avec la forme courte nom() {...}.

2.1 Préparation et Git (5 min)

- Se placer dans TPWeb, créer le sous-dossier TP7 et le dépôt Git ; sur GitHub, créer un dépôt **vide** nommé TP7 et le relier :

```
mkdir TP7; cd TP7; git init; git branch -M main
git remote add origin <URL_du_dépôt>
```

- Ouvrir le dossier dans VS Code et créer le dossier js.

2.2 Partie 1 — Le club informatique (65 min)

Note

Deux décimales : x.toFixed(2) garde exactement deux décimales (21.675.toFixed(2) → "21.68") mais renvoie une **chaîne** destinée à l'affichage, pas un nombre pour continuer les calculs.
Gabarit de chaîne : entre accents graves ` `, une chaîne peut contenir des expressions ${...} : `Bonjour ${prenom}` affiche la même chose que "Bonjour " + prenom (c'est la f-string de Python).
Enchaîner : comme filter et map renvoient un tableau, on peut écrire membres.filter(...).map(...). La moyenne s'écrit notes.reduce((acc, n) => acc + n, 0) / notes.length.
Tableau d'objets : map, filter, find s'y appliquent de la même façon, la fonction de rappel reçoit **un objet à la fois** : membres.filter((m) => m.age >= 18).

Créer club.html (titre « Club », h1 « Le club informatique »), relié au script js/club.js avec defer. Les quatre exercices s'écrivent à la suite dans ce même fichier ; commencer chaque exercice par console.log("---- Exercice 1 ----") (puis 2, 3, 4).

Exercice 1 : Math et String (10 min)

La cotisation annuelle du club est de 25.5 DT ; les étudiants de 1ère année ont une réduction de 15 %.

- Déclarer const cotisation = 25.5;, calculer cotisationReduite (la cotisation moins 15 %) et l'afficher telle quelle, puis avec deux décimales (toFixed(2)).
- Demander le prénom avec prompt("Votre prénom ?") et répondre volontairement sArRa (espaces et casse fantaisiste). Ranger dans prenom la saisie sans espaces inutiles (trim), puis construire prenomPropre : première lettre en majuscule et le reste en minuscules ([0], toUpperCase, slice(1), toLowerCase). Afficher Prénom : Sarra.
- Construire email avec un **gabarit** `${...}@club.tn`, tout en minuscules, et l'afficher ; vérifier avec includes qu'il contient bien @.

Exercice 2 : Date (10 min)

- Reprendre l'exemple du cours « Nous sommes le ... » : déclarer const aujourdhui = new Date(); ainsi que les deux tableaux jours (["dimanche", "lundi", ..., "samedi"]) et mois (["janvier", ..., "décembre"]).
- Afficher Nous sommes le lundi 9 novembre 2026 (avec la date du jour) grâce à getDay(), getDate(), getMonth() et getFullYear(). Rappel : les mois et les jours de la semaine commencent à **0**.

Exercice 3 : Tableaux (20 min)

- Déclarer const ateliers = ["HTML", "CSS", "JavaScript", "Git"];. Afficher sa longueur, le premier et le dernier élément (sans écrire 3 en dur) ; ajouter "Bootstrap" avec push ; retirer le dernier avec pop ; tester la présence de "PHP" avec includes. Afficher ensuite la liste numérotée 1. HTML, 2. CSS... avec for...of et un compteur let, puis console.table(ateliers).
- Déclarer const notes = [12, 8, 15, 10, 17, 6, 2];. Afficher notes.sort() : l'ordre obtenu est-il celui attendu ? Pourquoi ? Trier correctement avec notes.sort((a, b) => a - b), puis afficher les notes sous la forme 2 - 6 - 8 - ... avec join.
- Calculer et afficher, chacun en **une instruction** : admises, les notes supérieures ou égales à 10 (filter) ; sur100, les notes converties sur 100 (map) ; somme (reduce) puis moyenne arrondie à deux décimales (length, toFixed).

Vérification
Exercice 1 : Cotisation réduite : 21.675 → 21.68 ; Prénom : Sarra ; Email : sarra@club.tn ; Contient @ : true. Exercice 2 : Nous sommes le lundi 9 novembre 2026 (date du jour).

Exercice 3 : 4, HTML, Git ; après push/pop le tableau a 4 éléments ; PHP : false ; notes.sort() affiche [10, 12, 15, 17, 2, 6, 8] (tri alphabétique) ; tri numérique 2 - 6 - 8 - 10 - 12 - 15 - 17 ; admises : [10, 12, 15, 17] ; sur100 : [10, 30, 40, 50, 60, 75, 85] ; somme : 70 ; moyenne : 10.00.

Cycle Git : git add . → git commit -m "Exercices 1 a 3 : Math, String, Date et tableaux" → git push -u origin main

Exercice 4 : Objets (20 min)

- Déclarer l'objet membre avec les propriétés prenom ("Sarra"), age (19), ateliers (["HTML", "CSS"]) et cotisationPayee (false). Afficher le prénom avec le point et l'âge avec les crochets ; ajouter la propriété email (comme en a) ; parcourir l'objet avec for...in pour afficher propriete : valeur sur chaque ligne.
- Ajouter à l'objet, dans sa déclaration, la méthode sePresenter() qui renvoie "Je suis Sarra, 19 ans, inscrite à 2 ateliers : HTML, CSS." (utiliser this et join), puis l'afficher.
- Déclarer const membres contenant trois objets (sans méthodes) : Sarra, 19 ans, cotisation payée ; Ali, 22 ans, non payée ; Yasmine, 18 ans, payée. Calculer et afficher, chacun en une instruction : la liste des prénoms (map) ; les prénoms des membres qui n'ont **pas** payé (filter puis map) ; le membre prénommé "Yasmine" (find), puis son âge ; enfin console.table(membres).

Vérification

Sarra / 19 ; for...in affiche 5 lignes (dont email) ; Je suis Sarra, 19 ans, inscrite à 2 ateliers : HTML, CSS.
Prénoms : ["Sarra", "Ali", "Yasmine"] ; Non payés : ["Ali"] ; Yasmine : 18 ans ; console.table affiche une colonne par propriété.

Documentation du dépôt et cycle Git final

Créer un fichier README.md (un titre et 2 lignes de description) et y expliquer, avec vos propres mots, la différence entre for...of et for...in (un exemple sur un tableau, un sur un objet).

Cycle Git : git add . → git commit -m "Exercice 4 : objets et README" → git push

Vérifier sur GitHub que l'historique affiche deux commits.

2.3 Partie 2 — À la maison : le palmarès du quiz (45 min)

Note

Durée entre deux dates : reunion - aujourdhui donne un nombre de **millisecondes** ; pour obtenir des jours, on divise par 1000 * 60 * 60 * 24 et on arrondit avec Math.ceil. **`find`, `some`, `every`** : voir le tableau de la partie 1.5 c) ; la fonction de rappel reçoit un objet à la fois ((r) => r.score < 5).

Cette partie reprend l'essentiel du TP : tableau d'objets, méthodes de tableau, objet avec this, Math et Date. Elle est à rendre **avant la prochaine séance**, par un troisième commit sur le dépôt TP7.

Exercice 5 : le palmarès du quiz

Créer palmares.html (titre « Palmarès », h1 « Palmarès du quiz »), relié à js/palmares.js, et y déclarer les résultats du quiz du TP6 :

```
const resultats = [
  { joueur: "Sarra", score: 7 },
  { joueur: "Ali", score: 10 },
  { joueur: "Yasmine", score: 4 },
  { joueur: "Mohamed", score: 8 },
  { joueur: "Ines", score: 5 }
];
```

- **Pourcentage.** Avec forEach, ajouter à chaque objet la propriété pourcentage (score sur 10 → sur 100), puis afficher resultats avec console.table.
- **Classement.** Trier resultats par score **décroissant** (sort avec une fonction de comparaison), puis afficher le palmarès avec forEach et son deuxième paramètre (l'indice), une ligne par joueur : 1. Ali — 10 / 10 (100 %).
- **Vérifications.** Afficher si tous les joueurs ont au moins 3 (every), si quelqu'un a obtenu 10 (some), et le premier joueur qui doit repasser le quiz, score strictement inférieur à 5 (find).
- **Statistiques.** Construire l'objet stats avec les propriétés nbJoueurs, moyenne (arrondie à une décimale : Math.round(x * 10) / 10), meilleur (le premier objet du tableau trié), prochaineReunion (new Date(2026, 11, 15, 14, 30)), joursRestants (nombre de jours entre aujourd'hui et la réunion), et une méthode afficher() qui écrit ces informations dans la console en utilisant this.

Vérification

Le palmarès commence par 1. Ali — 10 / 10 (100 %) et finit par 5. Yasmine — 4 / 10 (40 %) ; every → true, some → true, find → l'objet de Yasmine.
stats.moyenne vaut 6.8 ; stats.meilleur.joueur vaut Ali ; joursRestants dépend de la date du jour (37 le 9 novembre).

Cycle Git : git add . → git commit -m "Exercice 5 : palmares du quiz (maison)" → git push

L'historique du dépôt doit alors afficher trois commits.